

Dijkstra's Algorithm

Dijkstra's algorithm is used to find the **shortest path from one source vertex to all other vertices** in a **weighted graph**, where **all edge weights are non-negative**.

It works for:

- directed graphs
- undirected graphs

It does **not** work correctly if negative edge weights are present.

1. Idea behind the algorithm

Suppose you start from a source node S.

You keep:

- the current best known distance from S to every vertex
- a way to repeatedly choose the vertex with the **smallest tentative distance**

Once a node gets chosen as the minimum, its shortest distance is finalized.

2. Main concept

For every node:

- distance is initially ∞
- source distance is 0

Then:

1. Pick the unvisited node with the smallest distance.
2. Mark it settled/finalized.
3. Relax all its outgoing edges.

Relaxation means:

If

$\text{distance}[u] + \text{weight}(u, v) < \text{distance}[v]$

then update:

$\text{distance}[v] = \text{distance}[u] + \text{weight}(u, v)$

and remember u as the parent of v .

3. Step-by-step procedure

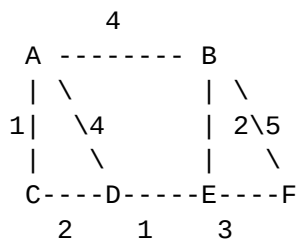
Let the source be S .

4. Set $\text{dist}[S] = 0$
5. Set all other distances to ∞
6. Put all nodes as unvisited
7. Repeatedly:
 - o choose the unvisited node u with the smallest $\text{dist}[u]$
 - o mark u visited
 - o for each neighbor v of u :
 - if v is unvisited and $\text{dist}[u] + w(u, v) < \text{dist}[v]$ then update $\text{dist}[v]$

Stop when all vertices are processed, or when the destination is finalized.

4. Example

Consider this graph:



Let us interpret edges as:

- $A \rightarrow B = 4$
- $A \rightarrow C = 1$
- $A \rightarrow D = 4$
- $C \rightarrow D = 2$
- $D \rightarrow E = 1$
- $B \rightarrow E = 2$
- $B \rightarrow F = 5$
- $E \rightarrow F = 3$

Find shortest distances from **A**.

Initial table

Vertex	Distance from A	Previous
A	0	-
B	∞	-
C	∞	-
D	∞	-
E	∞	-
F	∞	-

Visited = none

Step 1: Choose A

Smallest distance node is **A = 0**

Relax neighbors of A:

- B: $0 + 4 = 4 \rightarrow$ update B = 4
- C: $0 + 1 = 1 \rightarrow$ update C = 1
- D: $0 + 4 = 4 \rightarrow$ update D = 4

Vertex	Distance	Previous
A	0	-
B	4	A
C	1	A
D	4	A
E	∞	-
F	∞	-

Visited: A

Step 2: Choose C

Smallest unvisited distance is **C = 1**

Relax neighbors of C:

- D: $1 + 2 = 3$, current D = 4
Update D = 3

Vertex	Distance	Previous
A	0	-
B	4	A
C	1	A
D	3	C

E	∞	-
F	∞	-

Visited: A, C

Step 3: Choose D

Smallest unvisited distance is **D = 3**

Relax neighbors of D:

- E: $3 + 1 = 4 \rightarrow$ update E = 4

Vertex	Distance	Previous
A	0	-
B	4	A
C	1	A
D	3	C
E	4	D
F	∞	-

Visited: A, C, D

Step 4: Choose B

Smallest unvisited distance is **B = 4**

Relax neighbors of B:

- E: $4 + 2 = 6$, current E = 4 $\rightarrow$ no change
- F: $4 + 5 = 9 \rightarrow$ update F = 9

Vertex	Distance	Previous
A	0	-
B	4	A
C	1	A
D	3	C
E	4	D
F	9	B

Visited: A, C, D, B

Step 5: Choose E

Smallest unvisited distance is **E = 4**

Relax neighbors of E:

- $F: 4 + 3 = 7$, current $F = 9$
Update $F = 7$

Vertex	Distance	Previous
A	0	-
B	4	A
C	1	A
D	3	C
E	4	D
F	7	E

Visited: A, C, D, B, E

Step 6: Choose F

Smallest unvisited distance is $F = 7$

Done.

5. Final shortest distances from A

$A = 0$
 $B = 4$
 $C = 1$
 $D = 3$
 $E = 4$
 $F = 7$

6. Shortest paths

Using the Previous column:

- A to B: $A \rightarrow B$
- A to C: $A \rightarrow C$
- A to D: $A \rightarrow C \rightarrow D$
- A to E: $A \rightarrow C \rightarrow D \rightarrow E$
- A to F: $A \rightarrow C \rightarrow D \rightarrow E \rightarrow F$

7. Why it works

Dijkstra always chooses the node with the smallest temporary distance.

Because all edge weights are non-negative, no future path can produce a smaller value for that node after it is finalized.

That is why the chosen distance becomes permanent.

8. Pseudocode

```
Dijkstra(G, source):
    for each vertex v in G:
        dist[v] = infinity
        parent[v] = NIL
        visited[v] = false

    dist[source] = 0

    repeat n times:
        u = unvisited vertex with minimum dist
        if dist[u] == infinity:
            break

        visited[u] = true

        for each neighbor v of u:
            if visited[v] == false and dist[u] + weight(u,v) < dist[v]:
                dist[v] = dist[u] + weight(u,v)
                parent[v] = u
```

9. Efficient implementation using priority queue

The simple version finds the minimum vertex by scanning all nodes.

That gives time complexity:

- $O(V^2)$

Using a **min-priority queue / min-heap**:

- insert source with distance 0
- repeatedly extract minimum
- relax adjacent edges

Time complexity becomes:

- $O((V + E) \log V)$

This is the commonly used version.

10. Important note: negative weights

Dijkstra fails if a graph has negative edge weights.

Example idea:

If a node is finalized early, a later negative edge might produce a shorter path, which Dijkstra will miss.

For graphs with negative weights, use **Bellman-Ford algorithm**.

12. Small dry-run style format for exams

You can solve questions in table form like this:

Step	Chosen Vertex	Updated Distances
1	A	B=4, C=1, D=4
2	C	D=3
3	D	E=4
4	B	F=9
5	E	F=7
6	F	-

Final answer:

- shortest distances
- shortest path tree

13. Simple C-style algorithm

```
for (i = 0; i < n; i++) {
    dist[i] = INF;
    visited[i] = 0;
}
dist[source] = 0;

for (count = 0; count < n - 1; count++) {
    u = minDistance(dist, visited);
    visited[u] = 1;

    for (v = 0; v < n; v++) {
        if (!visited[v] && graph[u][v] &&
```

```
        dist[u] != INF &&  
        dist[u] + graph[u][v] < dist[v]) {  
            dist[v] = dist[u] + graph[u][v];  
        }  
    }  
}
```

14. Short definition for notes

Dijkstra's algorithm is a greedy algorithm used to find the shortest path from a given source vertex to all other vertices in a weighted graph with non-negative edge weights.